

WORKSHOP · 60 MINUTES · DEVOPS FOUNDATIONS

IPv4 vs IPv6

for DevOps Engineers

“Don’t panic. It’s just more numbers.”

Conceptual clarity · de-mystifying hex · security without NAT · a hands-on SR Linux lab

Why do we even care?

THE PROBLEM

IPv4 = ~4.3 billion addresses

2^{32} total — fewer than people on Earth. The global free pool ran dry years ago.

THE BAND-AIDS (AND THEIR COST)

NAT · RFC 1918 private space · CGNAT

- Per-packet translation latency
- Overlapping CIDRs & config sprawl
- Obscured source IPs → hard to audit
- NAT gateways = cost & throughput limits

THE FIX: STOP RATIONING

3.4×10^{38}

unique IPv6 addresses (340 undecillion)

Every grain of sand on Earth could have its own trillion addresses — and we'd barely dent the pool.

So every device, container, and pod can hold a **real, globally unique address** — no sharing, no translation.

Structure — and a mental model

IPv4 — like a local 7-digit number

192.168.1.1

32 bits · 4 base-10 octets · needs an operator (NAT) when two places want the same number

IPv6 — like a full international number

2001:0db8:0000:0000:
0000:8a2e:0370:7334

128 bits · 8 base-16 hextets · globally unique, no operator needed

READING THE TWO HALVES

2001:0db8:85a3:1f2c : 0000:0000:8a2e:7334

network / prefix (which LAN — like country + area code) interface / host (which device on it)

The cheat code: shrinking an address

Full: 2001:0db8:0000:0000:0000:8a2e:0370:7334

1

Drop leading zeros in each block

:0db8: → :db8:

2

Collapse one run of zero-blocks to ::

:0:0:0: → :: (once only!)

RESULT

2001:db8::8a2e:370:7334

Address types & the /64 convention

THE TYPES YOU'LL ACTUALLY MEET

Global Unicast `2000::/3`

public & routable — yes, on the host

Link-Local `fe80::/10`

auto, every interface; used by the plumbing

Unique Local `fd00::/8`

the v6 'private' range (≈ RFC 1918)

Multicast `ff00::/8`

replaces broadcast entirely

Loopback `::1/128`

localhost

Unspecified / default `:: · ::/0`

bind-to-all (::) · default route ::/0 (= 0/0)

THE /64 CONVENTION

Host / LAN subnets are /64 — because SLAAC needs the lower 64 bits for the host.

`2001:db8:acd2 : 0017 :: / 64`

◀ 48: global prefix ▶ ◀16: subnet▶
lower 64 bits = interface ID (the host)

...but it's a convention, not a law:

Point-to-point links → often **/127** (RFC 6164)

Loopbacks / single hosts → **/128**

DevOps payoff: for LANs, subnet sizing leaves your IaC — Terraform just templates a /64.

How a host actually gets its address

SLAAC

Stateless Address Auto-config

The host asks “any routers here?” (Router Solicitation); the router replies with the /64 prefix (Router Advertisement). The host builds its own address: prefix + its own interface ID. No server, no lease — it just works.

Hosts self-address at scale — no DHCP infra to run or break.

DHCPv6

Stateful assignment (when you want it)

A central server hands out addresses and options (DNS, etc.) and tracks who has what — just like IPv4 DHCP. Often paired with SLAAC: prefix via RA, DNS via DHCPv6.

Use when you need managed, audited, centrally-controlled assignment.

NDP

Neighbor Discovery — ARP's replacement

Runs over ICMPv6 + multicast. Finds neighbors on the link, detects duplicate addresses, and tracks reachability. There's no ARP and no broadcast in IPv6.

It's the “neighbor table” you'll inspect in the lab after a ping.

Typical boot flow: host sends RS → router answers RA (prefix) → host forms address via SLAAC → NDP checks it's unique & finds the gateway

What actually changes for infrastructure

Feature	IPv4	IPv6	DevOps impact
NAT	Mandatory for private nodes to reach out	Not needed — nodes hold global addresses	No NAT gateways, egress proxies, or CIDR overlap
Header	Variable length, options → more parsing	Fixed 40-byte header, streamlined	Faster, more predictable routing & firewalls
Subnetting	VLSM math: /24, /27, /30 ...	Standard /64 per LAN	Subnet sizing drops out of IaC templates
Security	IPsec optional, bolted on	IPsec standardized in design*	Cleaner baseline — but SGs/firewalls still rule

** IPsec was a MUST in early specs, relaxed to SHOULD in RFC 6434 — it's standardized & consistent, but not auto-on.*

Dismantling the fear: routable $\neq$ reachable

Core axiom: a globally unique address is not an open door. Reachability is a policy decision.

THE NAT FALLACY

“Behind NAT, attackers can't reach me.”

NAT was address rationing, never a security control. Hiding source IPs actually hurts you — audit logs, service meshes, and tracing a compromised container all fight the translation layer.

WHAT ACTUALLY PROTECTS YOU

Stateful firewalls & Security Groups drop unsolicited inbound by default — in v4 AND v6. A public IPv6 host is invisible if the policy drops inbound at the edge.

And brute-force scanning is dead:

a /64 = 18.4 quintillion hosts → **~580,000 yrs**
to scan at 1M/sec.

Privacy without NAT — and the dual-stack reality

KEEP THE 'PRIVATE SUBNET' PATTERN

IPv4: instance → NAT Gateway → internet

IPv6: instance → Egress-Only IGW → internet

Outbound the instance starts: allowed. Unsolicited inbound: dropped.
Same isolation as NAT — no rewrite, no bottleneck, and the gateway is free on AWS.

NOBODY FLIPS A SWITCH

The real world is dual-stack: v4 and v6 run side by side, services answering on both, for years — nobody turns IPv4 off overnight. Migration is incremental, and that's exactly what you'll build in the lab: one link, both families at once.



Cloud

AWS, Azure & GCP do IPv6 subnets, route tables & LBs natively.



Kubernetes

Dual-stack built in — Pods get both a v4 and a v6 address.



Your apps

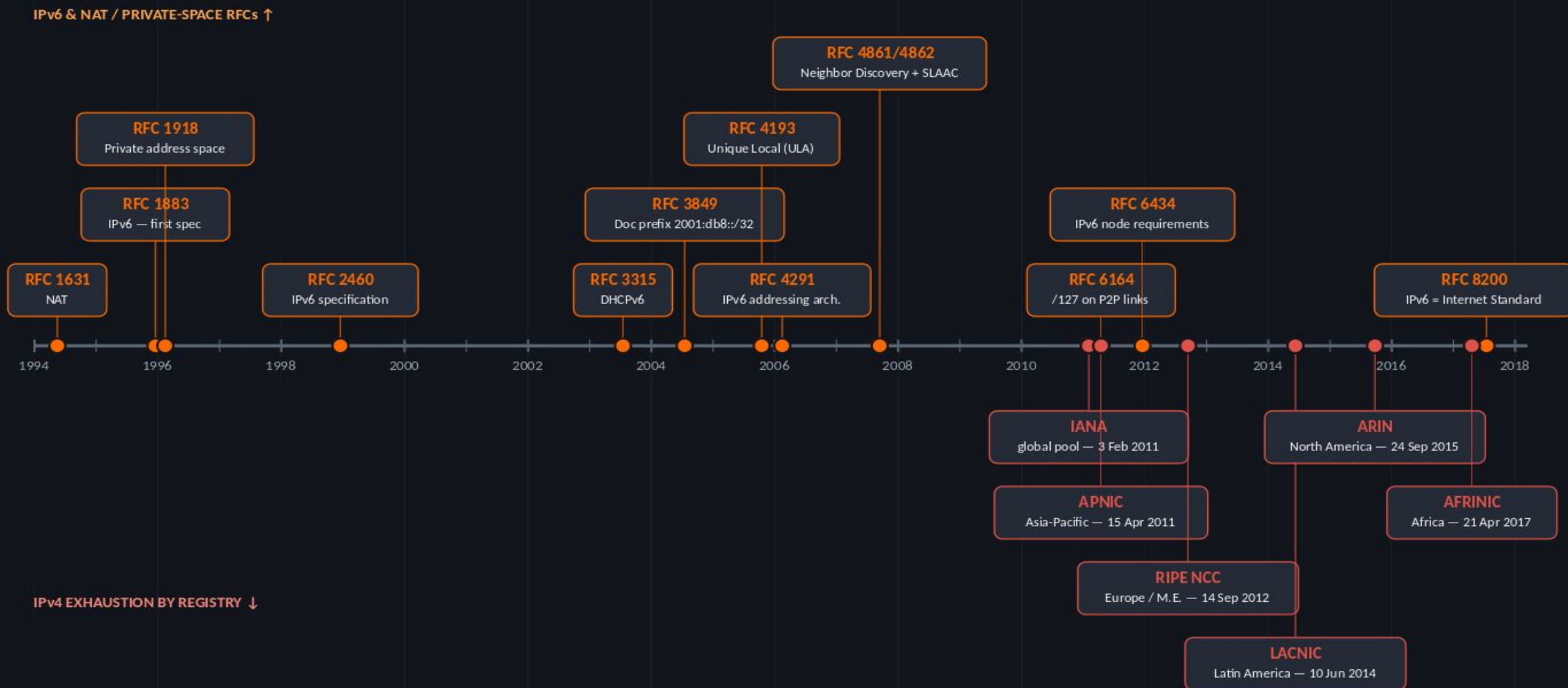
Cover BOTH families in every rule & listener, or you leave a gap.

IPv6 standards vs. IPv4 exhaustion

The relevant RFCs landed years before the regional registries actually ran out of IPv4

● IPv6 / NAT RFC published

● IPv4 pool exhausted (RIR)



Dates = when each registry reached its final-/8 / depletion phase and effectively stopped normal allocation; small reserved & recovered pools continued after (e.g. RIPE NCC made its last /22 allocations on 25 Nov 2019).

Lab: a dual-stack link you build yourself

TOPOLOGY · containerlab + SR Linux

```
server-a                server-b
  (100)                 (100)
   |                     |
[sr11]===e1-1===[sr12]

2001:db8:12::/64  +  192.168.12.0/30
```

Two SR Linux routers, one dual-stack link, two 'server' subnets to connect.

YOU WILL

- 1 Deploy the topology
- 2 Configure a dual-stack interface (v4 + v6)
- 3 Read the address SR Linux abbreviates for you
- 4 Inspect the neighbor (ND) table — ARP's replacement
- 5 Add static routes & ping across the fabric
- 6 Prove routable $\neq$ reachable with an ACL (bonus)

What to remember tomorrow

1

It's just more numbers

Two rules tame any address. Hex isn't the enemy.

2

No more NAT tax

Global addresses + egress-only gateways = privacy, no translation.

3

Routable $\neq$ reachable

Security is the firewall / Security Group, never IP hiding.

4

Design for dual-stack

Both families, every rule & listener — cloud and k8s already are.

RESOURCES

Cloud docs

AWS / Azure / GCP 'IPv6 VPC'

Kubernetes

kubernetes.io dual-stack

Lab

containerlab.dev · SR Linux docs

Standards

RFC 4291 · 4861 · 6434

Linux

ip -6 addr · ip -6 route